AlphaStock in Indian Stock Market
From Paper Understanding to Full Implementation

Dhairya Kantawala

IIT Bombay

April 21, 2026

Roadmap

1. Transition from Proposal to Execution
2. Data Pipeline for Indian Market
3. Model and Training
4. Results and Diagnostics
5. Takeaways

What the First Presentation Promised

- Original plan: implement AlphaStock for NIFTY 500 universe.

What the First Presentation Promised

- Original plan: implement AlphaStock for NIFTY 500 universe.
- Pipeline commitment: Bloomberg data $\rightarrow$ cleaning $\rightarrow$ feature engineering $\rightarrow$ RL model.

What the First Presentation Promised

- Original plan: implement AlphaStock for NIFTY 500 universe.
- Pipeline commitment: Bloomberg data $\rightarrow$ cleaning $\rightarrow$ feature engineering $\rightarrow$ RL model.
- Extension idea: replace paper's LSTM encoder with simpler MLP to reduce overfitting risk.

What the First Presentation Promised

- Original plan: implement AlphaStock for NIFTY 500 universe.
- Pipeline commitment: Bloomberg data $\rightarrow$ cleaning $\rightarrow$ feature engineering $\rightarrow$ RL model.
- Extension idea: replace paper's LSTM encoder with simpler MLP to reduce overfitting risk.
- Target metric remained Sharpe ratio (risk-adjusted performance), not only raw returns.

What Was Actually Built

- End-to-end data pipeline across `raw_data`, `semi_clean_data`, `final_clean_data`, and `normalised_data`.

What Was Actually Built

- End-to-end data pipeline across `raw_data`, `semi_clean_data`, `final_clean_data`, and `normalised_data`.
- Reusable helper API in `helper_functions.py` for consistent state/reward generation.

What Was Actually Built

- End-to-end data pipeline across `raw_data`, `semi_clean_data`, `final_clean_data`, and `normalised_data`.
- Reusable helper API in `helper_functions.py` for consistent state/reward generation.
- Full implementation notebook: `alphastock_mlp.ipynb`.

What Was Actually Built

- End-to-end data pipeline across `raw_data`, `semi_clean_data`, `final_clean_data`, and `normalised_data`.
- Reusable helper API in `helper_functions.py` for consistent state/reward generation.
- Full implementation notebook: `alphastock_mlp.ipynb`.
- Training, evaluation, and diagnostics plots exported for reporting.

Data Engineering Workflow

Bloomberg Export $\rightarrow$ Raw CSVs $\rightarrow$ Semi-cleaned Data
$\rightarrow$ Final Clean Data $\rightarrow$ Normalized Features $\rightarrow$ RL States/Rewards

- Universe aligned to Indian equities (NSE/BSE tickers present in cleaned datasets).

Data Engineering Workflow

$$\text{Bloomberg Export} \rightarrow \text{Raw CSVs} \rightarrow \text{Semi-cleaned Data}$$
$$\rightarrow \text{Final Clean Data} \rightarrow \text{Normalized Features} \rightarrow \text{RL States/Rewards}$$

- Universe aligned to Indian equities (NSE/BSE tickers present in cleaned datasets).
- Monthly feature creation and rolling normalization to avoid leakage.

Data Engineering Workflow

Bloomberg Export $\rightarrow$ Raw CSVs $\rightarrow$ Semi-cleaned Data
$\rightarrow$ Final Clean Data $\rightarrow$ Normalized Features $\rightarrow$ RL States/Rewards

- Universe aligned to Indian equities (NSE/BSE tickers present in cleaned datasets).
- Monthly feature creation and rolling normalization to avoid leakage.
- Reward definition kept consistent with long-short AlphaStock formulation.

Feature Engineering (Implemented)

```
def get_normalised_data(file):
    df = pd.read_csv("final_clean_data/"+file, index_col=0)
    df['ret_1m'] = np.log(df['price'] / df['price'].shift(1))
    df['ret_3m'] = np.log(df['price'] / df['price'].shift(3))
    df['mom_12m'] = df['price'] / df['price'].shift(12) - 1
    df['vol_3m'] = df['ret_1m'].rolling(3).std()
    df['vol_6m'] = df['ret_1m'].rolling(6).std()
    # ... volume, valuation, size and composite factors ...
    rolling_mean = df[features].rolling(24).mean().shift(1)
    rolling_std  = df[features].rolling(24).std().shift(1)
    df[features] = (df[features] - rolling_mean) / (rolling_std + 1e-8)
    df['R'] = (df['price'].shift(-1) / df['price'])
```

State and Reward Interface

```
def get_state_reward(time):
    month = months[time]
    state_i, reward_i = [], []
    for file in files_list:
        try:
            stock_data = pd.read_csv('normalised_data/'+file, index_col=0)
            s_i = stock_data.loc[month][features]
            r_i = stock_data.loc[month]['R']
            state_i.append(s_i)
            reward_i.append(r_i)
        except:
            continue
    return state_i, reward_i
```

Architecture Extension: MLP + Cross-Stock Attention

```
class MLPEncoder(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim, dropout=0.2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.BatchNorm1d(hidden_dim),
            nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(hidden_dim, hidden_dim), nn.BatchNorm1d(hidden_dim),
            nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(hidden_dim, output_dim)
        )
```

- Original AlphaStock idea preserved: learn stock representations then model cross-asset dependencies.

Architecture Extension: MLP + Cross-Stock Attention

```
class MLPEncoder(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim, dropout=0.2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.BatchNorm1d(hidden_dim),
            nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(hidden_dim, hidden_dim), nn.BatchNorm1d(hidden_dim),
            nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(hidden_dim, output_dim)
        )
```

- Original AlphaStock idea preserved: learn stock representations then model cross-asset dependencies.
- Practical change: LSTM replaced with compact MLP encoder.

Cross-Asset Attention (Core AlphaStock Mechanism)

```
class CrossStockAttention(nn.Module):
    def forward(self, r):
        Q = self.WQ(r); K = self.WK(r); V = self.WV(r)
        scores = torch.matmul(Q, K.T) / math.sqrt(self.d_model)
        alpha = F.softmax(scores, dim=1)
        A = torch.matmul(alpha, V)
        s = self.ws(A).squeeze(-1)
        return s
```

- Winner score for each stock is conditioned on all other stocks.

Cross-Asset Attention (Core AlphaStock Mechanism)

```
class CrossStockAttention(nn.Module):
    def forward(self, r):
        Q = self.WQ(r); K = self.WK(r); V = self.WV(r)
        scores = torch.matmul(Q, K.T) / math.sqrt(self.d_model)
        alpha = F.softmax(scores, dim=1)
        A = torch.matmul(alpha, V)
        s = self.ws(A).squeeze(-1)
        return s
```

- Winner score for each stock is conditioned on all other stocks.
- Enables relative ranking needed for long-short portfolio construction.

Sharpe-Oriented Loss

```
def compute_loss(R_list, H0=0.0):
    R = torch.stack(R_list)
    mu = R.mean()
    sigma = R.std() + 1e-6
    sharpe = mu / sigma
    cumulative = torch.cumprod(1 + R, dim=0)
    peak = torch.cummax(cumulative, dim=0).values
    drawdown = ((peak - cumulative) / (peak + 1e-8)).max()
    loss = -0.5 * (sharpe - H0) - 0.5 * mu * 12 + 0.2 * drawdown
    return loss
```

- Maintains paper objective while adding practical drawdown regularization.

Training Snapshot

- Parameters: 3,025 trainable.

Training Snapshot

- Parameters: 3,025 trainable.
- Input dimension: 14 engineered features.

Training Snapshot

- Parameters: 3,025 trainable.
- Input dimension: 14 engineered features.
- Sliding-window trajectory training on precomputed monthly states.

Training Snapshot

- Parameters: 3,025 trainable.
- Input dimension: 14 engineered features.
- Sliding-window trajectory training on precomputed monthly states.
- Holdout setup: last 24 months used for testing.

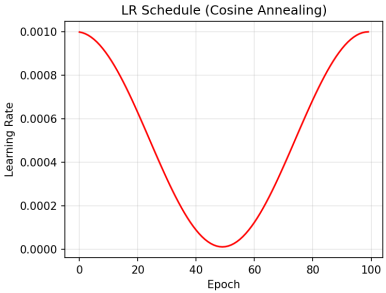
Training Snapshot

- Parameters: 3,025 trainable.
- Input dimension: 14 engineered features.
- Sliding-window trajectory training on precomputed monthly states.
- Holdout setup: last 24 months used for testing.

Observed Training Behavior

Epoch Sharpe moved from ~0.48 to ~1.78 with stable learning-rate schedule.

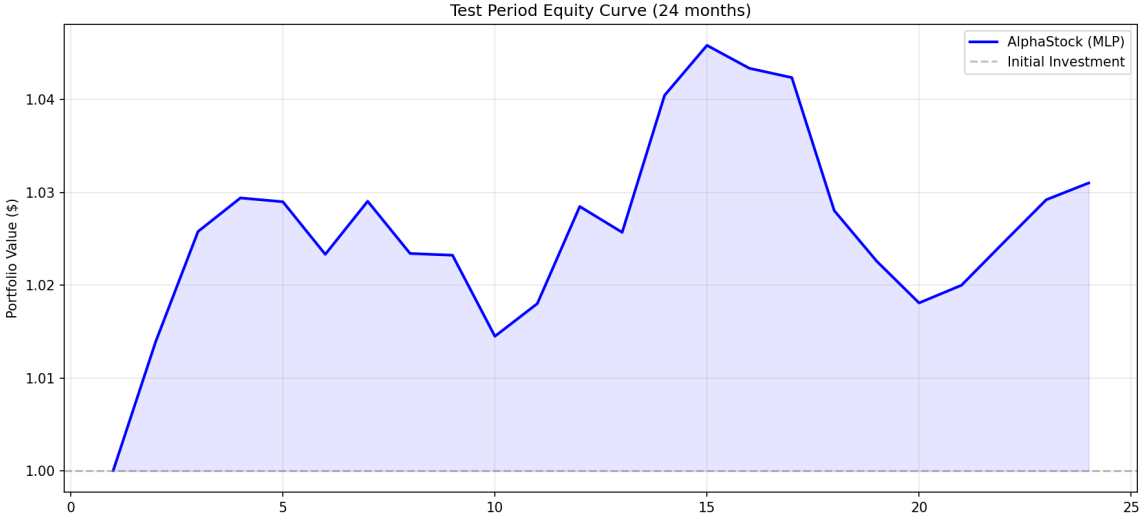
Training Curves



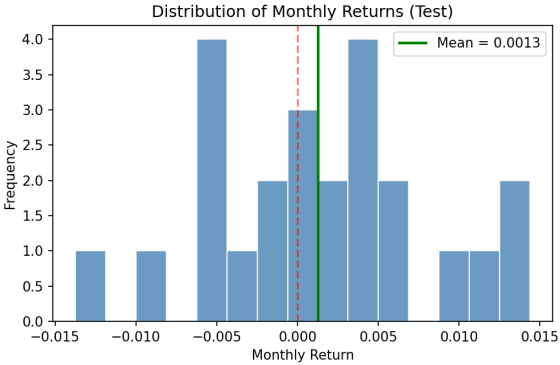
Test Performance (24-Month Holdout)

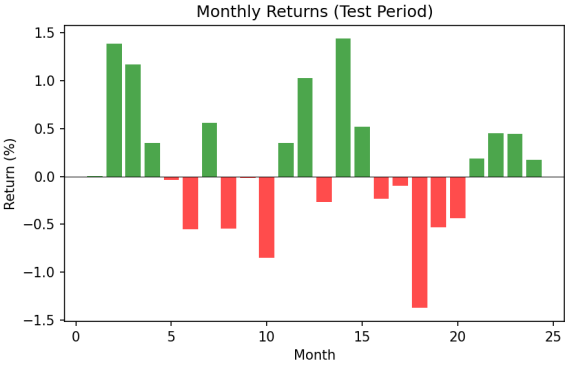
Metric	Value
Monthly Sharpe	0.1863
Annualized Sharpe	0.6454
Total Return	3.11%
Final Equity ($1 base)	1.0310
Annualized Return	1.54%
Max Drawdown	2.65%

Equity Curve and Drawdown

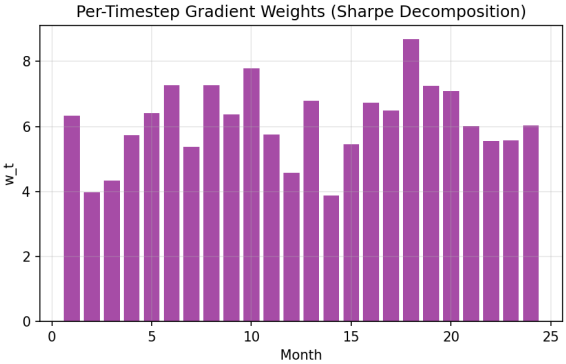


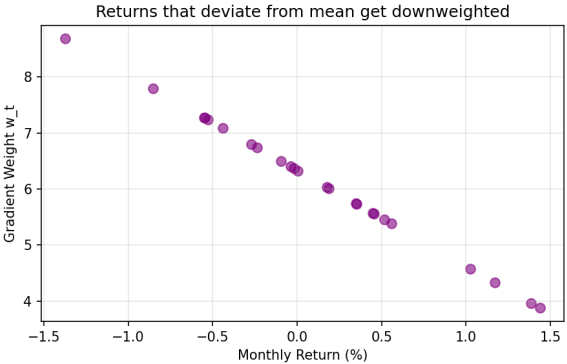
Return Distribution





Sharpe Gradient Weight Diagnostics





What This Project Demonstrates

- Smooth continuation of the original AlphaStock proposal into a full Indian-market implementation.

What This Project Demonstrates

- Smooth continuation of the original AlphaStock proposal into a full Indian-market implementation.
- Practical model variant (MLP encoder) can preserve core AlphaStock logic with lower parameter complexity.

What This Project Demonstrates

- Smooth continuation of the original AlphaStock proposal into a full Indian-market implementation.
- Practical model variant (MLP encoder) can preserve core AlphaStock logic with lower parameter complexity.
- Risk-aware training objective yields controlled drawdown with positive holdout performance.

What This Project Demonstrates

- Smooth continuation of the original AlphaStock proposal into a full Indian-market implementation.
- Practical model variant (MLP encoder) can preserve core AlphaStock logic with lower parameter complexity.
- Risk-aware training objective yields controlled drawdown with positive holdout performance.
- Codebase is reusable for future extensions: actor-critic, transaction costs, regime-aware features.

Next Steps

- I am not fully satisfied with the current results, but I loved this project and will keep improving it through the summer.

Next Steps

- I am not fully satisfied with the current results, but I loved this project and will keep improving it through the summer.
- Add explicit benchmark comparison (e.g., index and equal-weight long-short).

Next Steps

- I am not fully satisfied with the current results, but I loved this project and will keep improving it through the summer.
- Add explicit benchmark comparison (e.g., index and equal-weight long-short).
- Include transaction cost and turnover penalties.

Next Steps

- I am not fully satisfied with the current results, but I loved this project and will keep improving it through the summer.
- Add explicit benchmark comparison (e.g., index and equal-weight long-short).
- Include transaction cost and turnover penalties.
- Run multi-seed backtests and confidence intervals.

Next Steps

- I am not fully satisfied with the current results, but I loved this project and will keep improving it through the summer.
- Add explicit benchmark comparison (e.g., index and equal-weight long-short).
- Include transaction cost and turnover penalties.
- Run multi-seed backtests and confidence intervals.
- Expand to multi-horizon training windows and stress periods.

Thank You